DIAI Academy Backend Architecture

Project: DIAI Academy **Document Type:** Technical Architecture Reference **Audience:** Developers, Technical Stakeholders **Version:** 1.0 — Post-Refactor

---

Table of Contents

1. Folder Structure
2. Folder Responsibilities
3. File Responsibilities
   - 3.1 Root-Level Files
   - 3.2 core/
   - 3.3 services/
   - 3.4 routes/
   - 3.5 utils/
   - 3.6 ai/
4. Request Flow
   - 4.1 Hellen+ Chat Stream
   - 4.2 Learning Path Completion
5. Shared State
6. Dependency Flow
7. How to Add a New Feature
8. Architecture Diagram
9. Important Notes

---

1. Folder Structure

```
backend/
│
├── main.py                         ← App entry point (~79 lines)
├── database.py                     ← DB engine, session, get_db()
├── models.py                       ← SQLAlchemy ORM table definitions
├── schemas.py                      ← Pydantic request/response models
│
├── core/
│   └── state.py                    ← Shared in-memory state (embeddings, transcripts)
│
├── services/
│   ├── transcript_service.py       ← Parse & chunk transcript file
│   ├── embedding_service.py        ← Generate/cache text embeddings
│   ├── retrieval_service.py        ← Semantic similarity search
│   └── hellen_service.py           ← Hellen+ AI tutor logic
│
├── routes/
│   ├── auth_routes.py              ← Session & activity endpoints
│   ├── learning_path_routes.py     ← Learning path CRUD & progress
│   ├── analytics_routes.py         ← Analytics & ratings endpoints
│   └── hellen_routes.py            ← Hellen+ chat & streaming endpoints
│
```

```
├── utils/
│   └── helpers.py                  ← Shared utility functions & CSV lookups
│
├── ai/
│   ├── openai_service.py           ← Learning path AI recommendation
│   ├── guided_hellen_service.py    ← Guided Hellen+ onboarding messages
│   └── course_loader.py            ← CSV course/rating parser
│
├── routers/
│   └── scenarios.py                ← AI game scenario CRUD endpoints
│
└── data/
    ├── Video transcripts and Quizzes.txt
    ├── DIAI Academy eLearning details(Sheet1).csv
    ├── DIAI Academy eLearning details(Sheet2).csv
    └── transcript_embeddings.json   ← Disk cache for embeddings
```

---

2. Folder Responsibilities

`core/`

Holds shared global runtime state. The dicts defined here are populated once at startup and read by services on every request. Nothing else is stored here — no logic, no imports beyond typing.

`services/`

All business logic lives here. Services do the heavy lifting: parsing files, generating embeddings, searching transcripts, and building AI prompts. Routes call services; services never call routes.

`routes/`

FastAPI endpoint definitions only. Each file groups related endpoints into an `APIRouter`. Routes receive HTTP requests, delegate work to services, and return responses. They contain no business logic of their own.

`utils/`

Stateless helper functions shared across multiple parts of the app. Includes time parsing, learning path adaptation, module detection, rating computation, and CSV reference data loaded at import time.

`ai/`

Direct OpenAI/Azure API integrations, each scoped to a single job: generating learning path recommendations, producing guided teaching messages, and parsing course CSV data. These files were not changed during the refactor.

`routers/`

Legacy router directory. Contains `scenarios.py` (AI game scenario CRUD), which was already extracted before the main refactor. Kept separate for historical reasons.

`data/`

Static files only — no Python code. Contains the transcript text file, course metadata CSVs, and the embedding cache JSON. Nothing in this folder is modified at runtime except the embedding cache.

---

3. File Responsibilities

3.1 Root-Level Files

File	What It Does	Used By
`main.py`	Creates the FastAPI app, registers CORS middleware, includes all routers, and runs the startup sequence that populates `core/state`	Entry point — nothing imports it
`database.py`	Creates the SQLAlchemy engine and session factory; exposes `get_db()` as a FastAPI dependency	All route files via `Depends(get_db)`
`models.py`	Defines all SQLAlchemy ORM tables: `User`, `LearningPath`, `Response`, `LearningProgress`, `UserSession`, `ScreenActivity`, `UserRating`, `Scenario`, `ScenarioRating`	Route files when querying the database
`schemas.py`	All 18 Pydantic request/response models grouped by domain (learning path, session, Hellen+, ratings). Models with `orm_mode = True` serialize directly from ORM objects	All route files and `hellen_service.py`

3.2 `core/`

`core/state.py`

Declares four module-level mutable dicts that act as the application's in-memory data store.

Variable	Type	Contents
`embedded_chunks_map`	`Dict[str, List[Dict]]`	Transcript chunks with embedding vectors, keyed by submodule name
`quiz_by_topic`	`Dict[str, List[str]]`	Quiz question blocks, keyed by topic name
`transcript_data`	`Dict[str, List[Dict]]`	Raw parsed transcript segments, keyed by submodule name
`submodule_chunks_map`	`Dict[str, List[Dict]]`	Overlapping text chunks ready for embedding, keyed by submodule name

All four are empty at import time and filled by `main.py` at startup. Used by `hellen_service.py`.

3.3 `services/`

`services/transcript_service.py`

Parses `Video transcripts and Quizzes.txt` into structured Python dicts.

Function	Purpose
`parse_transcripts_with_timestamps()`	Splits the file into segments by submodule header
`parse_quiz_by_topic()`	Extracts quiz question blocks by topic name

`build_submodule_chunks()`	Produces overlapping text windows suitable for embedding

Used by: `main.py` at startup.

`services/embedding_service.py`

Calls Azure OpenAI's `text-embedding-3-small` model to turn text chunks into float vectors. Manages a disk cache so embeddings are not regenerated on every restart.

Function	Purpose
`_get_embedding(text)`	Single Azure API call → returns a float vector
`_cosine_similarity(a, b)`	Dot-product similarity between two vectors
`generate_chunk_embeddings()`	Embeds all chunks in a submodule map
`_cache_is_valid()`	Checks if the cache file is newer than the transcript file
`load_embedded_chunks_from_cache()`	Reads the JSON cache from disk
`save_embedded_chunks_to_cache()`	Writes the embedded map to disk
`load_or_generate()`	Convenience wrapper: load from cache or regenerate

Used by: `main.py` at startup, `retrieval_service.py` for per-request query embedding.

`services/retrieval_service.py`

Performs semantic search over `embedded_chunks_map`.

Steps per request:

1. Rewrites the user's query with GPT to improve match quality
2. Generates an embedding for the rewritten query
3. Scores all chunks in the relevant submodules by cosine similarity
4. Returns the top-k chunks filtered to within 75% of the best score

Used by: `hellen_service.py`.

`services/hellen_service.py`

The brain of the Hellen+ AI tutor. Contains the system prompt, intent classification, context retrieval, and message construction for both chat and practice modes.

Function / Constant	Purpose
`HELLEN_TUTOR_SYSTEM_PROMPT`	Large string constant defining Hellen+'s role, rules, and formatting
`_is_module_overview_intent()`	Detects "explain module / key takeaways" requests using combined keyword logic
`_resolve_submodule_names()`	Matches requested names against keys actually present in state
`_find_quiz_topic_match()`	Finds matching quiz blocks for the current message

`build_hellen_context_and_sources()`	Central dispatcher — returns a 4-tuple `(status, context, sources, chunks)`
`build_openai_messages()`	Assembles the messages list for standard chat requests
`build_practice_messages()`	Assembles the messages list for the adaptive Socratic practice mode

Return status values from `build_hellen_context_and_sources()`:

Status	Meaning
`"ok"`	Context found; proceed with LLM call
`"low_similarity"`	Best chunk score below threshold; no relevant transcript content
`"no_transcript"`	Submodule not loaded in state at all

Used by: `hellen_routes.py`.

3.4 `routes/`

`routes/auth_routes.py`

Manages user identity and activity tracking.

Endpoint	Method	Purpose
`/session/start`	POST	Creates a new session record; auto-creates user if not exists
`/session/end`	POST	Stamps logout time on the session record
`/activity/log`	POST	Records a screen visit with enter time, exit time, and duration

---

`routes/learning_path_routes.py`

Manages the full learning path lifecycle.

Endpoint	Method	Purpose
`/learning-paths/draft`	POST	Create or retrieve the user's current draft path
`/learning-path/{id}/response`	POST	Save a single questionnaire answer
`/learning-paths/{username}/draft`	GET	Fetch draft + saved responses
`/learning-path/{id}/complete`	POST	Complete questionnaire → trigger AI recommendation
`/learning-path/{id}/rename`	PATCH	Rename a completed path
`/learning-paths/{username}`	GET	List all completed paths for a user
`/learning-path/{id}`	GET	Fetch a single completed path
`/progress`	POST	Save submodule-level progress
`/progress`	GET	Fetch submodule-level progress

`routes/analytics_routes.py`

Twelve endpoints across user analytics, admin analytics, and ratings.

Endpoint	Method	Purpose
`/analytics/user/{username}`	GET	Full session + screen detail for one user
`/analytics/user/{username}/summary`	GET	Session count, total time, time per screen
`/ratings/learning-path/{id}`	GET	Average rating for a single path
`/ratings/user/{username}`	GET	All ratings submitted by a user
`/user-rating`	POST	Save or update a user's rating
`/user-ratings/{username}`	GET	All ratings by a user
`/analytics/admin`	GET	Platform-wide aggregates + per-user summary list
`/analytics/admin/filter-options`	GET	Available filter values for the admin view
`/analytics/admin/growth`	GET	User growth over time
`/analytics/admin/engagement`	GET	Session and screen engagement trends
`/analytics/admin/per-user`	GET	Per-user detailed metrics table
`/analytics/admin/feedback`	GET	User feedback/comments log

`routes/hellen_routes.py`

Four endpoints for the Hellen+ AI tutor feature.

Endpoint	Method	Mode	Purpose
`/hellen-chat`	POST	Synchronous	Full response returned in one JSON object
`/hellen-chat-stream`	POST	Streaming SSE	Streams `sources` → `token` → `...` → `done` events
`/hellen-practice-stream`	POST	Streaming SSE	Adaptive Socratic practice session
`/guided-hellen/next-question`	POST	Synchronous	Micro-teaching step for onboarding flow

---

3.5 `utils/`

`utils/helpers.py`

Function / Constant	Purpose
`parse_time_available()`	Converts `"2 hours"` string → integer minutes
`adapt_to_time()`	Scores and filters recommended submodules to fit the user's time budget
`detect_module_from_submodule()`	Maps a submodule name to its parent module (used in cross-module detection)

`compute_path_ratings()`	Averages submodule ratings from an `ai_summary` JSON blob
`ADMIN_USERNAMES`	`set` of admin email addresses used to gate admin analytics endpoints
`course_links_map`	Dict loaded from `Sheet2.csv`: learning path name → enrollment link
`submodule_ratings_map`	Dict loaded from `Sheet1.csv`: `(path, submodule)` → numeric rating

Used by: `learning_path_routes.py`, `analytics_routes.py`, `hellen_routes.py`.

3.6 `ai/`

`ai/openai_service.py`

Calls Azure OpenAI to generate a personalised learning path recommendation based on the user's questionnaire answers. Returns a structured JSON object with `selected_paths`, each containing modules and submodules with durations and reasoning.

Used by: `learning_path_routes.py`.

`ai/guided_hellen_service.py`

Generates a single micro-teaching step for the Guided Hellen+ onboarding flow. Each step uses a different instructional strategy:

Step	Strategy
Step 3	Introduce the subtopic briefly; ask if it makes sense
Step 4	Adapt based on previous answer — simplify if confused, deepen if understood
Step 5	Continue adaptive explanation; recommend creating a learning path

Used by: `hellen_routes.py`.

`ai/course_loader.py`

Reads `Sheet1.csv` and structures it into a nested `learning_path → module → submodule` hierarchy. Also provides `parse_rating()`, which extracts a numeric rating from freeform feedback strings like `"4 learners, rating 5.0, ..."`.

Used by: `ai/openai_service.py`, `utils/helpers.py`.

---

4. Request Flow

4.1 Hellen+ Chat Stream Flow

> *Endpoint:* `POST /hellen-chat-stream`

```
┌──────────────────────────────────────────────────────────────┐
│  Frontend sends:                                             │
│  POST /hellen-chat-stream                                    │
│  { module_name, submodule_names, message, history }          │
└──────────────────────┬───────────────────────────────────────┘
```

```
                        │
                        ▼
           hellen_routes.hellen_chat_stream()
                        │
                        ▼
       hellen_service._resolve_submodule_names()
       → Match requested names against core.state.embedded_chunks_map keys
       → Return only names that are actually loaded
                        │
                        ▼
       hellen_service.build_hellen_context_and_sources()
                        │
          ┌─────────────┴─────────────────┐
          │ Overview intent?              │ Regular question?
          │ ("explain module",            │
          │  "key takeaways", etc.)       │
          ▼                               ▼
  Collect ALL chunks             retrieval_service
  from state directly            .retrieve_relevant_chunks()
  (skip embeddings)                    │
          │                            ├── rewrite_query_for_retrieval()
          │                            │   [Azure GPT call]
          │                            │
          │                            ├── embedding_service._get_embedding()
          │                            │   [Azure Embedding call]
          │                            │
          │                            └── cosine_similarity against
          │                                state.embedded_chunks_map
          │
          └──────────────┬──────────────┘
                         │
                         ▼
             Returns (status, context, sources, chunks)

status == "ok"         → build_openai_messages(data, context)
status != "ok"
  + overview intent    → stream error message, skip LLM
  + regular question   → build fallback messages (no context)
                         │
                         ▼
       Open streaming POST to Azure OpenAI (stream=True)
                         │
                         ▼
       Yield SSE events to frontend:
         data: {"type": "sources", "sources": [...]}
         data: {"type": "token",   "content": "..."}  × N
         data: {"type": "done"}
```

4.2 Learning Path Completion Flow

> *Endpoint:* `POST /learning-path/{path_id}/complete`

```
┌──────────────────────────────────────────────────────────────┐
│  Frontend sends:                                             │
│  POST /learning-path/{id}/complete                           │
│  { job_function, experience, interests }                     │
└──────────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
    learning_path_routes.complete_learning_path()
                        │
                        ▼
    Load LearningPath + Responses from SQLite
    via get_db() dependency
                        │
                        ▼
    Build answers_text string from questionnaire responses
    (job function, experience, interests, goals, time available)
                        │
                        ▼
    ai/openai_service.generate_learning_recommendation()
    → Azure GPT call
    → Returns JSON: { selected_paths: [ { learning_path, modules: [...] } ] }
                        │
                        ▼
    utils/helpers.parse_time_available()
    → "X hours" → integer minutes
                        │
                        ▼
    utils/helpers.adapt_to_time()
    → Score each submodule:
        +50 if foundational keyword + beginner user
        +50 if advanced keyword  + advanced user
        +40 if submodule name matches user's interest words
        +10 if duration ≤ 30 min
    → Sort by score, filter to fit within time budget
    → Recalculate total_minutes, estimated_weeks, weekly_load_hours
                        │
                        ▼
    Attach course enrollment links from course_links_map (Sheet2.csv)
                        │
                        ▼
    Save ai_summary (JSON), status="completed" to SQLite
                        │
                        ▼
    Return { message, recommended_path, ai_summary }
```

---

5. Shared State

Why It Is Needed

The transcript file contains hundreds of text segments. Generating embeddings for all of them requires hundreds of Azure API calls and takes several minutes. This work can only be done once — at server startup — not on every

request.

The result (chunks + float embedding vectors) must live somewhere that all services can access without re-loading or re-computing.

How It Works

`core/state.py` declares four plain Python dicts at module level:

```
# core/state.py

embedded_chunks_map: Dict[str, List[Dict]] = {}
quiz_by_topic:       Dict[str, List[str]]  = {}
transcript_data:     Dict[str, List[Dict]] = {}
submodule_chunks_map: Dict[str, List[Dict]] = {}
```

They start empty at import time. When any Python module does `from core import state`, it receives a reference to these exact dict objects.

How It Is Populated

`main.py` runs the startup sequence immediately when the server starts (before any request is served):

```
# main.py — startup sequence

_transcript_data_raw = transcript_service.parse_transcripts_with_timestamps(TRANSCRIPT_FILE)
state.transcript_data.update(_transcript_data_raw)          # fill in place ✓

state.submodule_chunks_map.update(
    transcript_service.build_submodule_chunks(_transcript_data_raw)
)

state.quiz_by_topic.update(
    transcript_service.parse_quiz_by_topic(TRANSCRIPT_FILE)
)

state.embedded_chunks_map.update(
    embedding_service.load_or_generate(                     # cache or Azure API
        state.submodule_chunks_map,
        EMBEDDING_CACHE_FILE,
        TRANSCRIPT_FILE
    )
)
```

> ***Key detail:*** *`.update()` fills the existing dict object in place. Any other module that already imported `state.embedded_chunks_map` is holding a reference to the same dict — it automatically sees the populated data.*

How Services Access It

```python
# services/hellen_service.py

from core import state

# At request time — no arguments needed:
chunks = state.embedded_chunks_map.get(submodule_name, [])
quiz_blocks = state.quiz_by_topic.get(topic_name, [])
```

No function arguments needed. No class instances. The module reference is the singleton.

---

6. Dependency Flow

The dependency arrows point in one direction only — from top (entry point) down to primitives.

```
main.py
  │
  ├── database.py          (no internal deps)
  ├── models.py         →  database.py
  ├── schemas.py           (no internal deps)
  ├── core/state.py        (no internal deps)
  │
  ├── services/
  │   ├── transcript_service.py    (no internal deps)
  │   ├── embedding_service.py     (no internal deps)
  │   ├── retrieval_service.py  →  embedding_service
  │   └── hellen_service.py     →  core/state
  │                              →  schemas
  │                              →  retrieval_service
  │
  ├── utils/helpers.py     →  ai/course_loader
  │
  └── routes/
      ├── auth_routes.py          →  schemas, models, database
      ├── learning_path_routes.py →  schemas, models, database, utils, ai/openai_service
      ├── analytics_routes.py     →  schemas, models, database, utils
      └── hellen_routes.py        →  schemas, services/hellen_service
                                  →  utils/helpers
                                  →  ai/guided_hellen_service
```

Why There Are No Circular Imports

Each layer only imports from layers below it:

Layer	Imports from
`routes/`	`services/`, `utils/`, `schemas`, `models`, `database`, `ai/`
`services/`	`core/state`, `schemas`, sibling services, `ai/`
`utils/`	`ai/` only

`core/`	Nothing internal
`ai/`	Nothing internal (only third-party: `requests`, `pandas`)
`database/models/schemas`	Nothing internal (except `models` → `database`)

No lower-level module ever imports a higher-level module. `main.py` sits at the very top and imports everything — but nothing imports `main.py`. This one-directional flow makes the import graph a tree rather than a cycle.

---

7. How to Add a New Feature

Where Does New Code Go?

What you are adding	Where it goes
New API endpoint	`routes/<domain>_routes.py` — add a `@router.post/get/patch` function
New business logic	`services/<name>_service.py` — create a new file if the domain is new
New request or response shape	`schemas.py` — add a Pydantic model
New database table	`models.py` — add a SQLAlchemy class; `Base.metadata.create_all()` runs at startup
Shared utility or helper	`utils/helpers.py` — add a standalone function
New AI / LLM call	`ai/<name>_service.py` — keep direct API calls isolated here
New CSV reference data	Load in `utils/helpers.py` at module level (same pattern as `course_links_map`)
New in-memory data loaded at startup	Add the dict to `core/state.py`; populate it in `main.py` startup block

---

Concrete Example — Adding a `/feedback` Endpoint

Step 1 — Schema (`schemas.py`)

```
class FeedbackRequest(BaseModel):
    username: str
    message: str

class FeedbackOut(BaseModel):
    id: int
    username: str
    message: str
    created_at: datetime

    class Config:
        orm_mode = True
```

Step 2 — ORM Table (`models.py`)

```python
class Feedback(Base):
    __tablename__ = "feedback"

    id         = Column(Integer, primary_key=True, index=True)
    username   = Column(String, ForeignKey("users.username"))
    message    = Column(Text, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
```

Step 3 — Route (`routes/feedback_routes.py`)

```python
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
import models
from database import get_db
from schemas import FeedbackRequest, FeedbackOut

router = APIRouter(tags=["feedback"])

@router.post("/feedback", response_model=FeedbackOut)
def submit_feedback(data: FeedbackRequest, db: Session = Depends(get_db)):
    entry = models.Feedback(username=data.username, message=data.message)
    db.add(entry)
    db.commit()
    db.refresh(entry)
    return entry
```

Step 4 — Register in `main.py`

```python
from routes import feedback_routes
app.include_router(feedback_routes.router)
```

---

8. Architecture Diagram

Component Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                        FRONTEND (React)                          │
└──────────────────────────────┬───────────────────────────────────┘
                               │  HTTP / SSE
                               ▼
┌──────────────────────────────────────────────────────────────────┐
│                          main.py (FastAPI)                       │
│  CORS  │  Router registration  │  Startup sequence               │
└──────┬──────────┬──────────────┬────────────────┬────────────────┘
   │          │              │                │                │
   ▼          ▼              ▼                ▼                ▼
auth      learning      analytics         hellen          scenarios
routes    path routes    routes            routes          (routers/)
```

```
   |          |              |                |
   |          |              |                ▼
   |          |              |        services/hellen_service
   |          |              |                |
   |          |              |         ┌──────┴──────────┐
   |          |              |         ▼                 ▼
   |          |              |  retrieval_service    core/state
   |          |              |         |            (in-memory dicts)
   |          |              |         ▼                 ▲
   |          |              |  embedding_service        |
   |          |              |                     main.py populates
   |          |              |                     at startup via:
   |          |              |                      - transcript_service
   |          └──────────────┴────────────────────- embedding_service
   |                         |
   ▼                         ▼
database.py            ai/openai_service
(SQLite via            (Azure GPT)
 SQLAlchemy)
   ▲
   |
models.py (ORM tables)
schemas.py (Pydantic)
utils/helpers.py
```

Startup Sequence

```
Server starts
     |
     ▼
models.Base.metadata.create_all()   → ensure DB tables exist
     |
     ▼
transcript_service.parse_transcripts_with_timestamps()
     |  → state.transcript_data     ← filled
     ▼
transcript_service.build_submodule_chunks()
     |  → state.submodule_chunks_map ← filled
     ▼
transcript_service.parse_quiz_by_topic()
     |  → state.quiz_by_topic        ← filled
     ▼
embedding_service.load_or_generate()
     |  cache valid? → load from disk
     |  cache stale? → call Azure Embedding API for each chunk → save to disk
     |  → state.embedded_chunks_map  ← filled
     ▼
Server ready to accept requests ✓
```

Hellen+ Intent Routing Logic

```
Incoming message
       │
       ▼
_is_module_overview_intent(message)?
       │
   YES │                       NO
       ▼                       ▼
Collect ALL chunks       retrieve_relevant_chunks()
from state directly      (cosine similarity search)
(no Azure call)                 │
       │                best_score < 0.35?
       │                   │          │
       │                  YES         NO
       │                   ▼          ▼
       │            status =      status =
       │         "low_similarity"   "ok"
       │                   │          │
       └───────────────────┴──────────┘
                           │
                           ▼
                 status == "ok" → use transcript context
                 status != "ok" → fallback or error message
```

---

9. Important Notes

Do Not Reassign State Dicts

Always use `.update()` to populate shared state — never reassign:

```
# WRONG — other importers won't see this new dict object
state.embedded_chunks_map = new_dict

# CORRECT — modifies the existing object in place
state.embedded_chunks_map.update(new_dict)
```

Never Import Routes Inside Services

Services sit below routes in the dependency graph. Importing a route from a service creates a circular import and will crash the server at startup. If a service needs request data, it must receive it as a function argument.

Keep `main.py` Minimal

`main.py` should only contain:

- FastAPI app creation
- CORS middleware configuration
- Router registration (`app.include_router(...)`)
- Startup sequence (populate `core/state`)

If you find yourself writing logic in `main.py` , it belongs in a service, a route, or a utility instead.

`get_db()` Is the Only Correct Way to Open a DB Session

Never instantiate `SessionLocal()` directly inside a route handler. Always use:

```
db: Session = Depends(get_db)
```

This guarantees the session is properly closed after each request — even if an exception is raised.

`models.py` vs `schemas.py`

These two files are distinct and serve different purposes:

File	Framework	Purpose	Sent over HTTP?
`models.py`	SQLAlchemy	Define database tables and relationships	No
`schemas.py`	Pydantic	Define what JSON looks like on the wire	Yes — validated automatically by FastAPI

Output schemas that set `orm_mode = True` can be constructed directly from SQLAlchemy ORM objects with no manual mapping.

End of document.